

RailGaadi Features

1. Live Train Tracking

Features

- Search by Train Number or Train Name
- Live Train Location on Interactive Map
- Current Station & Next Station
- Running Status (On Time / Delayed)
- ETA to Upcoming Stations
- Distance Covered & Remaining
- Journey Progress Indicator
- Last Live Update Timestamp
- Auto Refresh Live Status
- Share Live Journey Link

2. Immersive Journey Map

Features

- Full Screen Interactive India Map
- Smooth Animated Train Marker
- Complete Railway Route Visualization
- Completed vs Remaining Route
- Current Station Highlight
- Camera Follow Train Mode
- Zoom, Rotate & Pitch Controls
- Beautiful Dark Map Theme
- Route Glow Animation
- Smart Map Loading & Transitions

3. Journey Analytics

Transform train data into meaningful insights.

Features

- Journey Completion Percentage
- Live Delay Analysis

- Distance Analytics
- Elevation Profile Along Route
- Highest Elevation Reached
- Travel Timeline
- Station Arrival History
- Journey Statistics Dashboard
- Route Summary
- Last Updated Status

4. Smart Travel Companion

Features

- Current Station Weather
- Next Station Weather Forecast
- Destination Weather
- Temperature, Humidity & Wind
- Rain Forecast Along Route
- Nearby Rivers & Lakes
- Mountains & Ghats
- Bridges & Tunnels
- Monuments & Tourist Attractions
- Nearby Cities & Districts

5. Premium User Experience

Features

- White theme Interface (also generate a [design.md](#) file)
- Apple Maps Inspired Design
- Fluid Micro-Animations
- Animated Counters
- Skeleton Loading States
- Smooth Page Transitions
- Responsive Mobile Experience
- Recent Searches
- Favourite Trains
- Clean Minimal Dashboard

Api Used

Railradar
Maptiler + maplibre
Openweather
Opentopography
[turf.js](#)
overpass

Prompt for PRD

ROLE

You are a Staff Product Manager, Senior Software Architect, and Senior UX Designer.

Your task is to convert the feature list I provide into a complete, production-ready Product Requirements Document (PRD).

Assume this is a real startup building an MVP that can scale into a commercial SaaS product.

Do not ask me questions unless absolutely necessary. Infer missing details intelligently from the feature list and make reasonable product decisions.

YOUR TASK

Using ONLY the feature list below, create a comprehensive PRD.

If something isn't explicitly mentioned, design the best possible implementation based on modern product design and engineering best practices.

Do not simply repeat the features. Expand them into detailed product requirements.

FEATURES

""""

PASTE FEATURES HERE

""""

OUTPUT REQUIREMENTS

Generate a complete PRD containing the following sections.

1. Product Vision

- Product overview
- Problem statement
- Target audience
- Goals
- Success metrics

2. User Personas

- Primary users
- User goals
- Pain points

3. Functional Requirements

Expand every feature into detailed implementation requirements including:

- User stories
- User flows
- Edge cases
- Validation
- Empty states
- Loading states
- Error handling

4. Information Architecture

- Application structure
- Navigation
- Screens
- Layout hierarchy

5. UI/UX Specification

Describe every screen in detail.

Include:

- Layout
- Components
- Cards
- Buttons
- Search
- Filters
- Modals
- Drawers
- Tables
- Maps (if applicable)
- Charts (if applicable)

- Mobile responsiveness
- Dark mode
- Animations
- Micro interactions

The UI should feel like a premium startup product inspired by Apple, Linear, Stripe, Arc Browser and Notion.

6. Technical Architecture

Design a scalable architecture.

Include:

- Frontend architecture
- Backend architecture
- State management
- Service layer
- Provider layer
- API layer
- Folder structure
- Component structure
- Reusable hooks
- Utility functions

7. API Design

Infer all required APIs from the feature list.

For every API specify:

- Purpose
- Request flow
- Response structure
- Error handling
- Caching strategy
- Retry strategy

8. Data Models

Generate all required data models, interfaces and database schema (if needed).

9. Component Inventory

List every reusable component that should be built.

10. Performance Strategy

Include:

- Lazy loading
- Code splitting
- Memoization
- Image optimization
- API caching
- Virtualization
- Debouncing
- Infinite scrolling (where appropriate)

11. Security

Explain:

- Authentication
- Authorization
- Input validation
- API security
- Environment variables
- Rate limiting

12. Accessibility

Follow WCAG best practices.

13. Development Roadmap

Break the project into logical milestones from MVP to production.

IMPORTANT

Do NOT generate a short PRD.

Think like you're writing documentation for a team of senior engineers and designers.

Expand every feature into implementation-level detail.

Make sensible assumptions where required.

Recommend additional features only if they naturally improve the product.

The final output should be detailed enough that developers can immediately start building the application without needing another planning document.